

DATABASE FOR DEVELOPERS

Exam Project, EASV PB Software 2025

Synopsis

Polyglot Persistence in Practice: Designing and Implementing Database Solutions for the SynopsisSI Microservice E-commerce Platform

Klaus Hviid

28-05-2025



DATABASES FOR DEVELOPERS

Indhold

1. INTRODUCTION	2
DATA MANAGEMENT IN MODERN APPLICATIONS	2
EXPLORING POLYGLOT PERSISTENCE IN SYNOPSISIS	2
THESIS STATEMENT	2
SCOPE OF WORK.....	2
STRUCTURE OF THE PAPER	2
2. POLYGLOT PERSISTENCE: CONCEPT AND RATIONALE.....	3
ADVANTAGES IN A MICROSERVICE ARCHITECTURE	3
CHALLENGES OF POLYGLOT PERSISTENCE.....	3
3. CASE STUDY SYNOPSISIS: DATABASE CHOICES	4
LISTINGSERVICE & ORDERSERVICE	4
USERSERVICE	5
4. IMPLEMENTATION OF THE DATA ACCESS LAYER	5
EF CORE AS AN ABSTRACTION LAYER	6
TAILORING DBCONTEXT CONFIGURATIONS FOR MONGODB AND POSTGRESQL	6
CONSISTENT DATA ACCESS PATTERNS: REPOSITORY AND UNIT OF WORK	7
5. COMPARISON AND ANALYSIS.....	7
ADVANTAGES OF THE CHOSEN POLYGLOT STRATEGY IN SYNOPSISIS.....	7
CHALLENGES AND CONSIDERATIONS DURING IMPLEMENTATION.....	8
HOW SYNOPSISIS WOULD DIFFER WITH A SINGLE DATABASE TYPE.....	8
<i>If Only PostgreSQL Were Used:</i>	<i>9</i>
<i>If Only MongoDB Were Used:.....</i>	<i>9</i>
6. CONCLUSION	9
ASSESSMENT OF GOAL ACHIEVEMENT REGARDING THE THESIS STATEMENT	9
LESSONS LEARNED AND REFLECTIONS.....	10
FUTURE WORK.....	10

The GitHub repository is available here: <https://github.com/zosodk/SynopsisSI>

It is created as a collaborative project for two courses: Databases for Developers and System Integration.

This repository will also serve as a tool for the practical examination of the Distributed Large Systems course, which requires practical examples of the technologies covered in the course.

1. Introduction

Data Management in Modern Applications

Modern software applications, particularly those built with microservice architectures, often encounter diverse data management needs across different functional components. A one-size-fits-all database solution can lead to compromises in performance, scalability, or schema flexibility for certain parts of the system. This has given rise to the concept of polyglot persistence, where multiple, specialized database technologies are employed within a single application ecosystem to best serve the specific requirements of individual services or data types.

Exploring Polyglot Persistence in SynopsisSI

This paper explores the practical application of polyglot persistence through the case study of SynopsisSI, a backend system for a modern e-commerce platform designed for second-hand item transactions. The SynopsisSI project consciously adopts different database technologies for its core microservices—MongoDB for services requiring schema flexibility and document-oriented storage, and PostgreSQL for services benefiting from relational structure and strong transactional consistency for identity data.

Thesis Statement

The central investigation of this paper is:

How can a polyglot persistence strategy, utilizing MongoDB for flexible-schema services (ListingService, OrderService) and PostgreSQL for a relational-identity service (UserService), be effectively implemented using Entity Framework Core (EF Core) as an abstraction layer to meet the diverse data management requirements of a microservice-based e-commerce platform (SynopsisSI), and what are the practical benefits and trade-offs of this approach?

Scope of Work

This paper will focus on the database design and data access layer implementation for three core microservices within SynopsisSI: the ListingService, OrderService (both using MongoDB), and the UserService (using PostgreSQL). It will detail the rationale behind choosing these specific database technologies for each service, demonstrate how Entity Framework Core is used to interact with them, and discuss the mapping of domain entities to the respective database schemas. While the broader microservice architecture (API Gateway, inter-service communication) provides context, the primary emphasis here is on the database solutions and the application of polyglot persistence.

Structure of the Paper

Section 2 will define polyglot persistence and discuss its general rationale. Section 3 will present the SynopsisSI case study, detailing the database choices and domain modeling for each key microservice. Section 4 will discuss the implementation of the data access layer using EF Core for both MongoDB and PostgreSQL. Section 5 will compare and analyze the chosen strategy, and Section 6 will offer a conclusion and reflections.

2. Polyglot Persistence: Concept and Rationale

Polyglot persistence refers to the practice of using multiple different database technologies within a single application or system to store and manage data. Instead of relying on a single, general-purpose database to handle all data needs, this approach advocates for selecting the most appropriate data store for specific types of data or particular functional requirements of different components or services. For instance, a system might use a relational database for transactional user account data, a document database for flexible product catalog information, a graph database for social connections, and a key-value store for caching session data.

Advantages in a Microservice Architecture

The microservice architectural style is particularly well-suited for polyglot persistence. Since each microservice is designed to be independent and own its data, it can also choose its own database technology without imposing that choice on other services. This alignment offers several key advantages:

Optimized Data Storage: Each service can select a database that best matches the structure, volume, access patterns, and consistency requirements of the data it manages. For example, a service handling product listings with highly variable attributes benefits from a schema-flexible document database, while a service managing financial transactions might require the strong ACID guarantees of a relational database.

Improved Performance and Scalability: By using a database tailored to specific needs, services can often achieve better performance for their typical operations. Furthermore, different database technologies have different scaling characteristics, allowing each service's data tier to be scaled appropriately.

Increased Developer Productivity & Innovation: Teams can leverage the database technologies they are most familiar with or that offer the best features for their specific domain problem, potentially leading to faster development and more innovative solutions.

Enhanced Resilience: The failure or performance degradation of one service's database is less likely to directly impact other services that use different, isolated data stores.

Challenges of Polyglot Persistence

While offering significant benefits, adopting a polyglot persistence strategy also introduces challenges that must be managed:

Increased Operational Complexity: Managing, monitoring, backing up, and maintaining multiple different database technologies can be more complex and require a broader set of operational skills compared to a single database system.

Data Integration and Consistency: If there's a need for data to be consistent or aggregated across services using different database types, this can be challenging. It often requires application-level solutions, such as event-driven synchronization or dedicated data aggregation services, rather than relying on database-native distributed transactions (which are typically not feasible across heterogeneous databases).

Developer Skillset: Development teams need to be proficient in multiple database technologies or have access to specialized expertise.

Tooling and Infrastructure: Supporting multiple database types may require a wider range of tools for development, administration, and monitoring.

For SynopsisSI, the decision to use polyglot persistence (initially MongoDB and PostgreSQL) was made by weighing the benefits of data model suitability and service autonomy against these potential complexities, with the plan to manage them through clear service boundaries and appropriate integration patterns.

3. Case Study SynopsisSI: Database Choices

The SynopsisSI e-commerce platform, designed with a microservice architecture, provides a concrete illustration of applying a polyglot persistence strategy. The selection of database technology for each core microservice—ListingService, OrderService, and UserService—was carefully considered, driven by the unique data characteristics, anticipated query patterns, and specific consistency requirements inherent to each service's domain. This tailored approach ensures that each service can leverage a data store optimally suited to its function, rather than conforming to a one-size-fits-all database solution.

ListingService & OrderService

For both the ListingService, which manages product catalog information, and the OrderService, responsible for customer orders, MongoDB was selected as the database technology.

Interaction with MongoDB is facilitated through Entity Framework Core, utilizing the official MongoDB.EntityFrameworkCore provider. This choice was primarily motivated by the need for schema flexibility and the nature of the data these services handle.

The ListingService benefits significantly from MongoDB's document model. Second-hand items often have a wide array of differing attributes; for example, apparel might have size and material specifics, while electronics would detail model numbers and technical specifications. Storing these as flexible ItemSpecifics within a ListingItem document is straightforward in MongoDB, avoiding the rigidity and potential for many null columns that a relational schema might impose for such varied data. Furthermore, listings often include embedded information like GeoLocation for proximity searches and lists of image URLs, all of which map naturally to MongoDB's document structure. EF Core's "owned entity type" feature is used here to model these embedded relationships, allowing GeoLocation to be stored as a sub-document within each ListingItem.

Similarly, the OrderService handles Order entities that can be complex, containing lists of OrderItem details and AddressVO value objects for shipping information. MongoDB's document model allows an entire order, with all its constituent parts, to be stored as a single, coherent document. This can simplify data retrieval for complete order views and aligns well with how orders are often conceptualized as a single unit of information.

From a scalability perspective, MongoDB offers robust horizontal scaling via replica sets for high availability and read scaling, and sharding for distributing very large datasets across multiple replica sets. This ensures that these critical e-commerce services can grow to handle increasing numbers of listings, orders, and users. MongoDB's rich query language and diverse indexing capabilities, including geospatial indexes (essential for the ListingService's location-based search features on ListingItem.Location) and text indexes (for keyword searches on titles

and descriptions), are also key advantages. Within the ListingServiceDbContext and OrderServiceDbContext, entities are mapped to their respective MongoDB collections (e.g., `modelBuilder.Entity<ListingItem>().ToCollection("Listings")`), and LINQ queries from the application layer are translated into MongoDB queries by the EF Core provider.

UserService

In contrast to the document-oriented approach for listings and orders, the UserService in SynopsisSI utilizes PostgreSQL, a powerful open-source relational database. This service is responsible for managing user identity, authentication credentials, and core profile information. EF Core, with the `Npgsql.EntityFrameworkCore.PostgreSQL` provider, is used for data access.

The rationale for selecting PostgreSQL for the UserService centers on the nature of user data and the required transactional integrity:

- **Structured Data & Schema Integrity:** Core user identity attributes like username, email, and hashed passwords are well-defined and benefit from the strong schema enforcement and data typing provided by a relational database.
- **ACID Transactions:** Operations such as user registration (atomically creating a user record and potentially assigning default roles) and password updates demand strong transactional consistency, which PostgreSQL provides reliably.
- **Relational Capabilities for Roles/Permissions:** While the initial implementation might store roles as a list within the user entity, a relational database like PostgreSQL is well-suited for more complex role and permission management systems involving join tables and foreign key constraints, should this be required in the future.

Mature Security Ecosystem: Relational databases have a long history and a mature ecosystem of tools and best practices for securing sensitive user data.

In the UserServiceDbContext, the User entity is mapped to a "Users" table, potentially within a dedicated "identity" schema (e.g., `modelBuilder.Entity<User>().ToTable("Users", "identity")`). Owned types like AddressVO within the User entity are mapped to columns within the Users table. Unique constraints are defined for Email and Username fields. Schema management for this relational database is handled using EF Core Migrations. Commands such as `dotnet ef migrations add` and `dotnet ef database update` are used to generate and apply SQL scripts that evolve the database schema in a controlled manner, based on changes to the UserServiceDbContext and its entity configurations.

This strategic use of PostgreSQL for the UserService alongside MongoDB for the ListingService and OrderService exemplifies the polyglot persistence approach in SynopsisSI. It allows each service to leverage a data store whose characteristics (schema flexibility, transactional model, query capabilities, scalability patterns) are optimally aligned with its specific domain requirements and data management needs.

4. Implementation of the Data Access Layer

The SynopsisSI platform leverages Entity Framework Core (EF Core) 9 as a pivotal component in its data access strategy, bridging the gap between the C# domain models of its microservices and the distinct underlying database technologies—MongoDB and PostgreSQL. This choice provides a consistent and familiar data access paradigm for developers, even within a polyglot

persistence environment, while still allowing for database-specific optimizations where necessary.

EF Core as an Abstraction Layer

Across the different microservices in SynopsisSI, EF Core serves as a crucial abstraction layer. It enables application developers to work primarily with their C# domain entities and to express data retrieval and manipulation logic using Language Integrated Query (LINQ). This approach shields much of the application code from the specific query languages or APIs of MongoDB and PostgreSQL. The respective EF Core database providers (`MongoDB.EntityFrameworkCore` and `Npgsql.EntityFrameworkCore.PostgreSQL`) take on the responsibility of translating these LINQ expressions into native database queries. This abstraction not only streamlines development but also enhances the testability of the application logic by allowing the `DbContext` and associated repositories to be more easily mocked or substituted with in-memory implementations during unit testing.

Tailoring DbContext Configurations for MongoDB and PostgreSQL

Each microservice within SynopsisSI that requires data persistence (e.g., `ListingService`, `OrderService`, `UserService`) defines its own service-specific `DbContext`. While these contexts all inherit from `Microsoft.EntityFrameworkCore.DbContext` and expose `DbSet<T>` properties for their root entities, their internal configuration within the `OnModelCreating` method is tailored to the chosen database for that service.

For services utilizing MongoDB, such as the `ListingServiceDbContext` and `OrderServiceDbContext`, the EF Core configuration is designed to embrace the strengths of the document model. Entities are mapped to MongoDB collections, and EF Core's support for "owned entity types" translates seamlessly to storing related data as embedded BSON documents. This is particularly beneficial for `ListingItem` entities, which can have a flexible `ItemSpecifics` dictionary mapped to a dynamic sub-document, and for `Order` entities, where line items and shipping details are naturally part of the main order document. While EF Core defines a conceptual model, the inherent schema flexibility of MongoDB is preserved. Schema management in this context is less about rigid migrations and more about evolving the application's understanding and handling of document structures, with indexing being a key performance consideration managed either via EF Core configurations or directly on the MongoDB server.

Conversely, for the `UserService`, which employs PostgreSQL, the `UserServiceDbContext` is configured to enforce a relational structure. Entities like `User` are mapped to traditional tables within a defined schema (e.g., "identity"). Data types are strictly enforced, and constraints such as `IsRequired()`, `HasMaxLength()`, and unique indexes (e.g., for email and username) are defined using EF Core's Fluent API. Owned entity types, like `AddressVO` within the `User` entity, are mapped to columns within the owner's table, maintaining relational integrity. A critical aspect of using PostgreSQL with EF Core in SynopsisSI is the use of EF Core Migrations. These tools provide a version-controlled approach to scaffold and apply schema changes to the relational database as the domain model evolves, ensuring that the database schema remains synchronized with the application's expectations.

Consistent Data Access Patterns: Repository and Unit of Work

Despite the underlying differences in database technologies, SynopsisSI promotes a consistent approach to data access logic within each microservice through the disciplined application of the Repository and Unit of Work (UoW) patterns.

The Repository pattern abstracts the specifics of data querying and persistence for aggregate root entities. Each microservice's Application layer defines repository interfaces (e.g., `IListingRepository` in `ListingService.Application`, `IUserRepository` in `UserService.Application`) that outline the required data access operations. Concrete implementations of these interfaces then reside in the respective Infrastructure layers, utilizing the service-specific EF Core `DbContext` to perform the actual database interactions. This ensures that the application's command and query handlers remain decoupled from EF Core specifics and the choice of database.

Complementing the Repository pattern, each microservice employs a service-specific Unit of Work pattern, exposed via an `IUnitOfWork` interface defined in its Application layer. The concrete `UnitOfWork` implementation in the Infrastructure layer encapsulates the service's `DbContext`. This pattern is responsible for managing the lifecycle of the `DbContext` for a given operation, providing access to the relevant repositories, and, crucially, orchestrating the saving of all tracked changes via a single `SaveChangesAsync()` call. Furthermore, the `IUnitOfWork` provides methods for explicit transaction management (`BeginTransactionAsync`, `CommitTransactionAsync`, `RollbackTransactionAsync`). EF Core then translates these UoW transaction methods into the native transaction capabilities of the underlying database, whether it's MongoDB's multi-document ACID transactions (when used with replica sets) or PostgreSQL's standard ACID transactions.

This layered approach, with EF Core as the common data access framework and consistent use of Repositories and the Unit of Work pattern, allows SynopsisSI to effectively manage its polyglot persistence strategy, providing both tailored data storage solutions and a standardized development experience for data access logic across its microservices.

5. Comparison and Analysis

The adoption of a polyglot persistence strategy within the SynopsisSI e-commerce platform, specifically utilizing MongoDB for the `ListingService` and `OrderService`, and PostgreSQL for the `UserService`, offers distinct advantages while also presenting certain challenges. This section analyzes these aspects and compares the chosen approach against a hypothetical single-database architecture.

Advantages of the Chosen Polyglot Strategy in SynopsisSI

The primary advantage of employing multiple database technologies in SynopsisSI is the ability to tailor data storage precisely to the data's nature and its access patterns for each microservice.

The `ListingService`, for example, manages product listings which inherently have diverse and often unpredictable attributes, especially in a second-hand marketplace. MongoDB's schema-flexible document model excels here, allowing `ItemSpecifics` to be stored as dynamic key-value

pairs within a listing document without necessitating a rigid, pre-defined table structure with numerous nullable columns. This also simplifies the storage of related data like image URLs or tags as embedded arrays. Similarly, Order documents in the OrderService can naturally encapsulate complex structures including lists of ordered items and shipping details as embedded data, which aligns well with how orders are often processed and retrieved as a single unit.

For the UserService, PostgreSQL provides a structured, relational environment well-suited for managing core user identity information, including credentials that demand high integrity and well-defined relationships (e.g., future role and permission management). The strong ACID transactional guarantees of PostgreSQL are beneficial for sensitive operations like user registration and password updates.

This tailored approach can lead to performance benefits. MongoDB can offer high performance for use cases involving large volumes of semi-structured data or queries that align with its document model (e.g., retrieving a full listing document with all its embedded details in a single operation). PostgreSQL, on the other hand, is optimized for complex relational queries, joins, and transactional workloads typical of identity management. By matching the database to the service's primary workload, SynopsisSI aims for optimized performance characteristics where they matter most for each domain.

Challenges and Considerations during Implementation

While beneficial, the polyglot persistence strategy in SynopsisSI is not without its challenges and considerations.

A primary consideration is increased operational complexity. Managing, monitoring, backing up, and ensuring the high availability of two distinct database technologies (MongoDB and PostgreSQL) requires a broader set of operational skills and potentially more complex infrastructure tooling compared to a single-database system. Each database has its own scaling characteristics, security configurations, and maintenance procedures.

The need for developer skillset diversity also arises. Development teams working across different microservices, or even within a single service that might indirectly rely on data patterns from another, need to be comfortable with the concepts and query languages of multiple database systems. While EF Core provides an abstraction layer, understanding the underlying database's behavior is still crucial for optimization and troubleshooting.

Finally, while microservice architecture aims for service autonomy and loose coupling, data consistency and integration across services using different databases require careful thought. If, for example, a ListingService needed to display user information alongside a listing, it would typically retrieve this via an API call to UserService rather than a direct database query or join. For maintaining consistency (e.g., if user deletion needed to reflect on their listings), event-driven patterns (as discussed for system integration) become essential for propagating changes asynchronously. This avoids distributed transactions across heterogeneous databases, which are generally complex and brittle.

How SynopsisSI Would Differ with a Single Database Type

Contrasting the chosen polyglot approach with a hypothetical single-database architecture for SynopsisSI highlights the trade-offs made:

If Only PostgreSQL Were Used:

The ListingService would face challenges in elegantly managing the diverse and evolving ItemSpecifics for different product types. This would likely necessitate solutions like EAV (Entity-Attribute-Value) patterns, JSONB columns, or many sparsely populated columns, each with its own complexities in terms of querying, indexing, and schema evolution. While PostgreSQL's JSONB capabilities are powerful, they might not offer the same native document-centric development experience as MongoDB for this specific use case. Retrieving a complete listing with all its variable attributes might also involve more complex queries compared to fetching a single document from MongoDB.

If Only MongoDB Were Used:

The UserService could certainly store user profiles and credentials in MongoDB. MongoDB supports ACID transactions on replica sets, which could handle user registration. However, managing strongly defined relationships (like user roles and permissions, if they become complex) and enforcing schema consistency for core identity data might feel less natural or require more application-level validation compared to a relational model. While MongoDB is highly capable, the established patterns and tooling around relational databases for identity management are very mature.

By adopting polyglot persistence, SynopsisSI attempts to leverage the "best tool for the job" for each core service, aiming for an optimal balance of schema flexibility, transactional integrity, performance, and scalability tailored to the distinct needs of its different domains. This architectural decision acknowledges that the data management requirements of a user identity system can differ significantly from those of a product catalog or an order processing system within the same e-commerce platform.

6. Conclusion

The SynopsisSI e-commerce platform has effectively demonstrated the application of a polyglot persistence strategy within a microservice architecture. By selecting MongoDB for the ListingService and OrderService, the platform leverages the flexibility of a document model to handle diverse product attributes and complex order structures. Simultaneously, the use of PostgreSQL for the UserService provides the strong consistency, relational integrity, and transactional guarantees necessary for managing sensitive user identity and credential data. Entity Framework Core 9 served as a valuable abstraction layer, enabling a consistent data access approach across these different database technologies, while still allowing for database-specific configurations and optimizations within each service's infrastructure layer.

Assessment of Goal Achievement regarding the Thesis Statement

The core thesis question:

How can a polyglot persistence strategy, utilizing MongoDB for flexible-schema services and PostgreSQL for a relational-identity service, be effectively implemented using EF Core to meet the diverse data management requirements of a microservice-based e-commerce platform?

Has been addressed through the practical implementation within SynopsisSI. The project successfully showcases that distinct database technologies can be chosen and implemented for individual microservices based on their specific data characteristics and operational needs. The use of EF Core, with its respective MongoDB and Npgsql providers, proved to be a viable approach for interacting with both database types from a .NET environment, offering a balance between developer productivity and database-specific capabilities.

Lessons Learned and Reflections

The implementation highlighted several key learnings. Firstly, the "database per service" model, a cornerstone of microservice design, naturally lends itself to polyglot persistence, granting significant autonomy to each service in its data management choices. Secondly, while EF Core provides a useful abstraction, a good understanding of the underlying database's strengths and weaknesses (e.g., MongoDB's document structure and query patterns vs. PostgreSQL's relational integrity and transactional models) remains crucial for effective data modeling and optimization. Finally, the operational aspect of managing multiple database types, even in a containerized development environment, underscores the need for robust configuration management and clear database schemas (or schema flexibility guidelines for NoSQL stores) for each service.

Future Work

The polyglot persistence strategy in SynopsisSI is foundational and can be expanded as the platform evolves. Future work could involve integrating additional specialized data stores for new microservices. For example:

A SearchService could leverage Elasticsearch for its advanced full-text search, faceting, and aggregation capabilities, requiring event-driven data synchronization from the ListingService.

A potential RecommendationsService or social feature might benefit from a graph database (like Neo4j) to manage complex relationships between users, listings, and interactions. These additions would further enrich the polyglot nature of SynopsisSI, presenting new data modeling, access, and integration challenges and opportunities, providing further practical case studies for advanced database concepts.